pfi

Python for Fixed Income

COURSE 9 · DEPLOYMENT

CI with GitHub Actions

WEEK 1 · DAY 3

Created by **Jeff Lenamon**

This course is educational content created in a personal capacity. Nothing here is investment advice or a recommendation to buy or sell any security. All portfolio data is synthetic.

Today: CI with GitHub Actions

Today

1. What CI is, and where Actions came from
2. The workflow file, written and read as a table
3. The first bash of the series, line by line
4. YAML's traps, and check_workflow
5. Linux is not Windows: line endings, file names
6. The matrix: two Pythons, fail-fast
7. The course's runs: red, red, green
8. Secrets in CI, and how the log hides them

Three functions

`workflow_text`

the course's workflow YAML

`workflow_steps`

any workflow as a table

`check_workflow`

the problems that still run

No notebook cell reaches GitHub: the runs are shown from their masked transcripts.

What CI is

push

a commit or a PR

→

runner

a fresh Linux machine

→

steps

install, then pytest

→

result

a tick or a cross

Where it came from

16 Oct 2018: "Future of Software: Developers at the center of the universe", Jason Warner (limited public beta)

8 Aug 2019: "GitHub Actions now supports CI/CD, free for public repositories", Nat Friedman

Source: github.blog, both posts, read 2026-10-12

A green run proves the repository holds everything the tests need.

GitHub Actions: free on public repos, standard runners (docs.github.com/en/billing/concepts/product-billing/github-actions, read 2026-10-12).

The workflow file

`.github/workflows/tests.yml`

```yaml
name: tests
on: [push, pull_request]
jobs:
  test:
    runs-on: ubuntu-latest
    strategy:
      matrix:
        python-version: ["3.12", "3.13"]
    steps:
      - uses: actions/checkout@v7
      - uses: actions/setup-python@v7
        with:
          python-version: ${{ matrix.python-version }}
      - run: python -m pip install --upgrade pip
      - run: python -m pip install -r requirements.txt
      - run: python -m pytest -q test_bondmath.py
```

```
step 1  actions/checkout@v7
step 2  actions/setup-python@v7
step 3  python -m pip install --upgrade pip   (bash)
step 4  python -m pip install -r requirements.txt   (bash)
step 5  python -m pytest -q test_bondmath.py   (bash)
```

Read it top down

`on`	every push and every pull request
`runs-on`	a Linux runner, ubuntu-latest
`matrix`	the job once per Python
`uses / run`	an action at a tag, or a command

The first bash of the series

`python -m pip install --upgrade pip` PowerShell at home: the same line	upgrade the installer on the fresh Python
`python -m pip install -r requirements.txt` PowerShell at home: the same line	install every pinned line of the file
`python -m pytest -q test_bondmath.py` PowerShell at home: the same line	run the tests; a failure turns the run red

```
exit code: 0
40 passed in 1.30s
```

Default shell on a Linux runner: bash -e {0}, stopping at the first command that fails.

YAML's traps

What yaml.safe_load reads

```
python-version: [3.10, 3.13, '3.12']
{'python-version': [3.1, 3.13, '3.12']}
on: [push]
{True: ['push']}
```

Quote every version. Every one of these still runs on GitHub; some go green.

```
the course's workflow: []
an action on a branch:
  step 1: actions/checkout@main is not pinned
versions without quotes:
  python-version 3.1 is read as a number
  python-version 3.13 is read as a number
pushes only, no pytest:
  no pull_request trigger
  no step runs pytest
a secret printed:
  step 6: a run line prints a secrets. expression
```

Line endings: the demo repo, fixed

```
i/crlf w/crlf attr/ .gitignore
i/lf w/lf attr/ README.md
i/crlf w/crlf attr/ bondmath.py
i/crlf w/crlf attr/ bondmath_excel_reference.csv
i/crlf w/crlf attr/ bondmath_excel_reference_tvm.csv
i/crlf w/crlf attr/ test_bondmath.py
```

before: day 2's merge, five files CRLF

```
i/lf w/lf attr/text=auto eol=lf .gitignore
i/lf w/lf attr/text=auto eol=lf README.md
i/lf w/lf attr/text=auto eol=lf bondmath.py
i/lf w/lf attr/text=auto eol=lf bondmath_excel_reference.csv
i/lf w/lf attr/text=auto eol=lf bondmath_excel_reference_tvm.csv
i/lf w/lf attr/text=auto eol=lf test_bondmath.py
```

after: every file LF, under the rule

```
.gitattributes:  * text=auto eol=lf
git add --renormalize .
```

6 files changed, 1466 insertions(+), 1465 deletions(-)

A rule in the repository beats a setting on each laptop.

File names: case matters on Linux

Windows

Bondmath.csv and bondmath.csv: one file

Linux

Bondmath.csv and bondmath.csv: two files

```
on this machine: 40 passed in 1.30s
the planted copy: ['Bondmath_Excel_Reference.csv']
the work folder:  []
```

Write a file's name exactly as it is on disk, and keep names lower case.

The matrix

test (3.12)

the Python of a ZeroGPU Space (day 10)

test (3.13)

the course's Python (your 3.13.8)

```
Successfully set up CPython (3.13.16)
['python-version: ["3.11", "3.12", "3.13"]']
read back: ['3.11', '3.12', '3.13']
```

Fail-fast: when one job fails, GitHub cancels the others still running.

The course's runs: red, red, green

Run 1: red

```
Could not open requirements file:
[Errno 2] No such file or directory:
'requirements.txt'
```

fix: requirements.txt, pytest

Run 2: red

```
ModuleNotFoundError: No module named
'numpy'
```

fix: numpy added

Run 3: green

```
40 passed in 2.14s
```

both Pythons

```
createdAt                                displayTitle event conclusion
2026-10-12T10:31:41Z    Run the tests on every push and pull request  push    failure
2026-10-12T10:32:12Z Add the requirements file the workflow installs  push    failure
2026-10-12T10:33:33Z               Add numpy, which one test imports  push    success
```

A red run is a message about the repository; the log names the reason.

Secrets in CI, Excel, and Git

${{ secrets.NAME }}: read by name, never typed

"automatically redacts the contents of all GitHub secrets that are printed to workflow logs"

"Because there are multiple ways a secret value can be transformed, this redaction is not guaranteed."

Source: docs.github.com/en/actions/concepts/security/secrets, read 2026-10-12

Excel side by side: none today

No workflow, no runner, no test on every change.

```
Run the tests on every push and pull request
.github/workflows/tests.yml | 16 ++++++++++++++++
requirements.txt |  2 ++
2 files changed, 18 insertions(+)
```

Never print a secret. On a public repo, anyone can read the logs.

The AI check

The docstring is the prompt. The deploy code that comes back runs and looks finished.

1

Docstring

Units, conventions, and what the code must never let through: what you asked for.

2

Code back

Written for the course in the style of an AI answer. Read it.

3

Test first

Plant the case the docstring promises to catch. Run it: it fails.

4

Fix

Find the rule it broke, fix it, run the test again.

Today: the check only asks whether python-version exists, so [3.10, 3.13] passes and the runner gets 3.1.

Code that runs is not the same as code that is right.

What you can do now

- CI: a fresh Linux machine runs your tests
- workflow_text, workflow_steps, check_workflow
- Read the bash; quote every version
- .gitattributes, and names in their exact case
- Read a red run's log, fix the repository

Week 1, Day 4 What the Server Installs